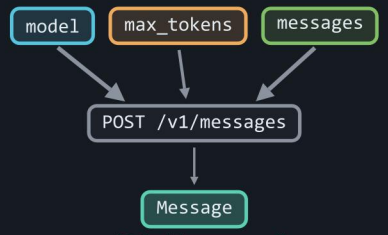
LEVEL 01

LLM API FUNDAMENTALS

Project 01 · Steps 1–2 · The First Call & Memory · 115 XP available

EQUIPPED MODEL
`claude-haiku-5-5`
$0.10 in · $0.50 out per 1M tokens

① THE CALL



an SDK object, not a dict

3 required M's:
Model · Max_tokens · Messages
One call = one HTTP request.
Every call starts from **zero**: no memory.

```
response = client.messages.create(
    model=MODEL,
    max_tokens=4000,
    messages=messages,
)
```

+10 XP

② TYPES: YOU SEND DICTS, YOU GET OBJECTS

YOU SEND ▸ `dict` plain Python

```
messages = [
  {"role": "user",
   "content": "My name is Sam"},
  {"role": "assistant",
   "content": response.content},
]
```

You build these. Read with **brackets**:
`msg["role"]`
`MessageParam` is only a type hint;
at runtime it is a plain dict.

◂ YOU GET `Message` pydantic object

```
response: Message
├ .content: list[ContentBlock]
│  ├ ThinkingBlock .thinking
│  └ TextBlock .text  ← answer
├ .stop_reason: str  "end_turn"
└ .usage: Usage
   ├ .input_tokens: int
   └ .output_tokens: int
```

Read with **dots**: `block.text`
`ContentBlock` is a union of block types:
check `block.type` before reading `.text`.

RULES OF THE GAME
✓ `block.text` ✗ `block["text"]` → TypeError: not subscriptable
✓ `json.dumps([b.model_dump() for b in response.content])` → JSON-safe dicts
✗ `json.dumps(response.content)` → TypeError: not JSON serializable
✓ `{"role": "assistant", "content": response.content}` → the SDK takes its objects back

+25 XP

③ TOKENS = MONEY



1 word → 3 tokens

≈ ¾ word per token · billed **in** and **out**
8-word prompt → **22** input tokens (overhead)

cost = in ÷ 1M × in price
+ out ÷ 1M × out price

OUTPUT PRICE PER 1M
Opus 5.5 $20.00
Haiku 5.5 $0.50 (40× less)

SAME QUESTION, REAL RUNS
Opus 142 out → $0.002928
Haiku 73 out → $0.000039
75× cheaper: price + Haiku skipped thinking
Output costs **5×** input per token.

+15 XP

④ BOSS: max_tokens

`max_tokens = 1000`



✓ `end_turn`

`max_tokens = 20`



✗ `max_tokens` · 0 words · still billed 20

`max_tokens` caps thinking + text together.
A ceiling, not a target: unused is free.
Strategy: set it high (4000) and
always check `stop_reason`.

+20 XP

⑤ STATUS: stop_reason

`end_turn`
finished normally → safe to use

`max_tokens`
hit your cap → answer incomplete

`tool_use`
wants YOU to run a tool (Project 06)

`refusal`
the model declined the request

`model_context_window_exceeded`
history too big for the model (step 5)

Rule: check it **before** trusting `content`.
Type: a plain `str` → compare with ==

+10 XP

⑥ MEMORY = A LIST





per turn: linear
session total: quadratic
(x = turns)

1 run = 1 session = 1 list.
`quit` → list gone. The API stores nothing.
Keep it: save `model_dump()` output to JSON.

+20 XP

⑦ TRAP CARDS

⚠ **`content[0]` isn't always text**
Opus thought first; Haiku skipped it.

⚠ **`output_tokens` includes thinking**
You pay for words you never see.

⚠ **`temperature` → 400 error**
Current models use effort (step 3).

⚠ **"\$" in a Python string**
SyntaxWarning. Just write $.

⚠ **Same prompt ≠ same output**
142 vs 168 tokens: measure many runs.

+15 XP

PROJECT 01 S1 S2 S3 S4 S5 S6

UNLOCKED ★ First Call ★ Read the Response ★ Priced a Call ★ Beat the Boss ★ Gave Claude Memory

NEXT QUEST ▸ Step 3: Streaming